

UQTL – Unified Query Translation Layer

Sprachübergreifende Standardisierung der Query"-Übersetzung
von Python, Java (Streams) und C#/.NET (LINQ) nach SQL

Stephan Epp

Datenbankstandards, Sprachdesign, API-Normierung

6. März 2026

Zusammenfassung

Die vorliegende Arbeit definiert den **Unified Query Translation Layer (UQTL)**, einen offenen technischen Standard zur einheitlichen Übersetzung von datenabfragenden Ausdrücken aus den Programmiersprachen Python, Java (Stream API) und C#/.NET (LINQ) in äquivalente SQL-Anweisungen. Das zentrale Ziel des Standards ist die *semantische Äquivalenzgarantie*: Gleichartige Abfragen, unabhängig von der Quellsprache formuliert, erzeugen stets dasselbe normalisierte SQL-Statement.

Der Standard umfasst eine formale Grammatik für den sprachunabhängigen intermediären Ausdrucksbaum (Intermediate Query Tree, IQT), vollständige Übersetzungsregeln für alle gängigen Query-Operatoren (Filterung, Projektion, Aggregation, Sortierung, Joins, Fensterfunktionen und Unterabfragen), sowie Konformitätsstufen (Conformance Levels CL-1 bis CL-4) für Implementierungen. Umfangreiche Vergleichsbeispiele belegen die Anwendbarkeit des Standards auf reale Datenbankszenarien.

Schlagwörter: Query Translation, SQL-Generierung, LINQ, Python ORM, Java Streams, Datenbankabstraktion, AST-Normierung, Interoperabilität

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation und Problemstellung	4
1.2	Zielsetzung des Standards	4
1.3	Abgrenzung	5
1.4	Aufbau der Arbeit	5
2	Theoretische Grundlagen	7
2.1	Relationale Algebra und SQL	7
2.2	Query-Abstraktion in modernen Programmiersprachen	7
2.2.1	Python – SQLAlchemy Core & ORM	7
2.2.2	Java – Stream API und JPA Criteria API	8
2.2.3	C# – LINQ (Language Integrated Query)	8
2.3	Bestehende Ansätze und deren Limitierungen	9
2.4	Formale Grundbegriffe	10
3	Der Intermediate Query Tree (IQT)	12
3.1	Überblick und Architektur	12
3.2	IQT-Knotentypen	12
3.2.1	Knoten-Spezifikation im Detail	12
3.3	Normalisierungsregeln des IQT	16
4	Übersetzungsregeln und Operatoren	17
4.1	Operator 1: Filterung (WHERE / HAVING)	17
4.2	Operator 2: Projektion (SELECT)	17
4.3	Operator 3: Sortierung (ORDER BY)	17
4.4	Operator 4: Begrenzung (LIMIT / OFFSET)	17
4.5	Operator 5: Aggregation (GROUP BY)	17
4.6	Operator 6: Joins	17
4.7	Operator 7: Fensterfunktionen (OVER)	17
4.8	Operator 8: Unterabfragen und CTEs	26
4.9	Operator 9: Mengenoperationen	26
5	Praxisbeispiele	27
5.1	Datenmodell für die Beispiele	27
5.2	Beispiel 1: Einfache Filterung und Projektion	28
5.3	Beispiel 2: Aggregation mit GROUP BY und HAVING	29
5.4	Beispiel 3: INNER JOIN mit mehreren Bedingungen	30
5.5	Beispiel 4: LEFT OUTER JOIN und NULL-Behandlung	31
5.6	Beispiel 5: Fensterfunktionen (OVER PARTITION BY)	32
5.7	Beispiel 6: Unterabfragen (Subqueries)	34

5.8	Beispiel 7: Komplexer Multi-Join mit Aggregation	36
5.9	Beispiel 8: EXISTS-Unterabfrage	38
5.10	Beispiel 9: Common Table Expressions (WITH)	40
5.11	Beispiel 10: Vollständige Geschäftslogik-Abfrage	42
6	Konformitätsstufen	44
6.1	CL-1: Basiskonformität	44
6.2	CL-2: Standardkonformität	44
6.3	CL-3: Erweiterte Konformität	45
6.4	CL-4: Vollständige Konformität	45
7	Referenzimplementierung	46
7.1	Architektur einer UQTL-konformen Implementierung	46
7.2	Pseudocode des IQT-Builders (Python-Frontend)	46
7.3	Pseudocode des SQL-Codegenerators	47
7.4	Konformitätstests	49
8	Diskussion	51
8.1	Stärken des UQTL-Standards	51
8.2	Grenzen und Limitierungen	51
8.3	Erweiterungen und zukünftige Arbeit	52
9	Zusammenfassung	52
A	Vollständige IQT-Grammatik (EBNF)	54
B	Operatormapping-Referenztafel	57

1 Einleitung

1.1 Motivation und Problemstellung

Moderne Softwaresysteme sind in hohem Maße polyglott: Unterschiedliche Dienste einer verteilten Anwendungslandschaft werden in verschiedenen Programmiersprachen entwickelt, greifen jedoch häufig auf gemeinsame relationale Datenbanksysteme zu. Python-basierte Datenpipelines, Java-Microservices und C#-Backend-Dienste formulieren dabei konzeptuell identische Datenbankabfragen – verwenden jedoch vollständig unterschiedliche Ausdrucksmittel.

Betrachten wir die simple Frage „*Alle aktiven Nutzer, deren Umsatz im vergangenen Quartal über 1 000 Euro lag, sortiert nach Name*“: In Python (SQLAlchemy), Java (Stream API mit JPA Criteria) und C#/.NET (LINQ to Entities) entstehen syntaktisch völlig unterschiedliche Ausdrücke, die jedoch – bei korrekter Implementierung – denselben SQL-Ausdruck erzeugen sollten. In der Praxis tun sie dies *nicht*: ORMs und Query-Builder unterschiedlicher Sprachen und Frameworks produzieren inkompatible, unvollständige oder semantisch abweichende SQL-Ausgaben [2, 4, 5].

Daraus entstehen konkrete Probleme in der Softwareentwicklung:

- **Inkonsistente Abfrageergebnisse:** Zwei Dienste, die dieselbe Geschäftslogik ausdrücken, erhalten unterschiedliche Daten.
- **Erschwerte Code-Reviews:** Entwickler müssen den mentalen Overhead betreiben, Query-Ausdrücke verschiedener Sprachen zu verstehen und zu vergleichen.
- **Migrations-Risiken:** Bei der Portierung von Java- nach Python-Diensten oder umgekehrt kann keine mechanische Äquivalenzprüfung erfolgen.
- **Fehlende Testbarkeit:** Ohne definiertes Normalformat ist die automatisierte Verifikation der Query-Äquivalenz nicht möglich.

1.2 Zielsetzung des Standards

Der **Unified Query Translation Layer (UQTL)** adressiert diese Problematik durch die Definition eines umfassenden technischen Standards, der folgende Ziele verfolgt:

1. **Semantische Äquivalenzgarantie:** Ausdrücke derselben logischen Abfrage in Python, Java und C# werden stets in dasselbe normalisierte SQL-Statement übersetzt.
2. **Intermediäre Darstellung:** Ein sprachunabhängiger *Intermediate Query Tree*

(*IQT*) dient als gemeinsame Zwischenrepräsentation.

3. **Vollständige Operatorabdeckung:** Alle wesentlichen Query-Operatoren (Filter, Projektion, Aggregation, Sortierung, Joins, Fensterfunktionen, Unterabfragen) sind standardisiert.
4. **Erweiterbarkeit:** Der Standard definiert Erweiterungspunkte für datenbankspezifische Dialekte (PostgreSQL, MySQL, Oracle, MSSQL).
5. **Konformitätsstufen:** Vier Konformitätsstufen (CL-1 bis CL-4) erlauben schrittweise Implementierungen.

Kerndefinition des Standards:

Für jede Menge logisch äquivalenter Query-Ausdrücke Q_{py} , Q_{java} , Q_{cs} aus Python, Java bzw. C# gilt unter UQTL:

$$\text{SQL}(Q_{py}) \equiv_{\sigma} \text{SQL}(Q_{java}) \equiv_{\sigma} \text{SQL}(Q_{cs})$$

wobei $\equiv_{\sigma}$ die semantische SQL-Äquivalenz unter einer gegebenen Datenbankschema-Instanz σ bezeichnet.

1.3 Abgrenzung

Der UQTL-Standard ist *kein* vollständiges ORM-Framework und definiert *keine* Laufzeitbibliothek. Er spezifiziert:

- Die formale Syntax und Semantik des Intermediate Query Tree (IQT)
- Übersetzungsregeln von den drei Quellsprachen in den IQT
- Übersetzungsregeln vom IQT nach ANSI SQL:2016
- Normalisierungsregeln für eindeutige SQL-Ausgaben
- Konformitätsanforderungen für Implementierungen

Explizit nicht Teil des Standards sind: Datenbankzugriff, Verbindungsverwaltung, Entity-Mapping, Caching-Strategien sowie dialektspezifische Optimierungen (diese können als optionale Erweiterungen definiert werden).

1.4 Aufbau der Arbeit

2 legt die theoretischen Grundlagen dar: relationale Algebra, Query-Abstraktionen und bestehende Ansätze. 3 definiert den Intermediate Query Tree formal. 4 spezifiziert alle Übersetzungsregeln. 5 demonstriert den Standard anhand umfangreicher Praxisbeispiele.

6 definiert die Konformitätsstufen. 7 beschreibt Referenzimplementierungen. 8 diskutiert Limitierungen und Erweiterungen. 9 fasst die Ergebnisse zusammen.

2 Theoretische Grundlagen

2.1 Relationale Algebra und SQL

Die theoretische Basis für die Query-Übersetzung bildet die *relationale Algebra* nach Codd [1]. Alle im UQTL-Standard behandelten Query-Operatoren lassen sich auf eine endliche Menge algebraischer Grundoperationen zurückführen.

Definition 2.1 (Relationale Grundoperationen). Gegeben seien Relationen R, S über einem Schema $\mathcal{R}$. Die relationalen Grundoperationen sind:

$\sigma_p(R)$	Selektion (Filterung) mit Prädikat p
$\pi_A(R)$	Projektion auf Attributmenge $A \subseteq \mathcal{R}$
$R \times S$	Kartesisches Produkt
$R \cup S$	Vereinigung
$R - S$	Differenz
$R \bowtie_p S$	Join mit Prädikat p
$\gamma_{A,F}(R)$	Gruppierung nach A mit Aggregatfunktionen F

SQL ist eine deklarative Anfragesprache, die über diese algebraischen Grundoperationen hinausgeht und um Fensterfunktionen, rekursive Abfragen (CTE), Mengenoperationen sowie datenbankspezifische Erweiterungen ergänzt wird. ANSI SQL:2016 [2] bildet die Ziel-Spezifikation des UQTL-Standards.

2.2 Query-Abstraktion in modernen Programmiersprachen

Alle drei im UQTL-Standard behandelten Sprachen implementieren das Konzept der *Query Comprehension* oder *monadischen Query-Komposition*:

2.2.1 Python – SQLAlchemy Core & ORM

Python verfügt über keine native Query-Syntax in der Sprache selbst. Der De-facto-Standard für relationale Abfragen ist **SQLAlchemy**, dessen *Core*-Schicht einen Expression Language API bereitstellt. Abfragen werden als verkettete Methodenaufrufe oder als ORM-Ausdrücke formuliert [3]:

```
1 # Stil 1: SQLAlchemy ORM (Session.query -- Legacy)
2 users = session.query(User)\
3     .filter(User.active == True)\
4     .order_by(User.name)\
5     .all()
6
```

```

7 # Stil 2: SQLAlchemy 2.0 Select-Style (modern)
8 stmt = select(User)\
9     .where(User.active == True)\
10    .order_by(User.name)
11 users = session.execute(stmt).scalars().all()
12
13 # Stil 3: Pandas / pandasql (fuer Datenpipelines)
14 import pandasql as ps
15 users = ps.sqldf("SELECT * FROM users WHERE active=1 ORDER BY name")

```

Listing 1: Python SQLAlchemy – Query-Stilarten

2.2.2 Java – Stream API und JPA Criteria API

Java bietet seit Version 8 die **Stream API** für funktionale Datenverarbeitung auf Collections [7]. Für Datenbankabfragen wird die **JPA Criteria API** oder Frameworks wie **QueryDSL** verwendet:

```

1 // Stil 1: Stream API (In-Memory-Verarbeitung)
2 List<User> users = allUsers.stream()
3     .filter(u -> u.isActive())
4     .sorted(Comparator.comparing(User::getName))
5     .collect(Collectors.toList());
6
7 // Stil 2: JPA Criteria API (Datenbankabfrage)
8 CriteriaBuilder cb = em.getCriteriaBuilder();
9 CriteriaQuery<User> cq = cb.createQuery(User.class);
10 Root<User> root = cq.from(User.class);
11 cq.select(root)
12     .where(cb.isTrue(root.get("active")))
13     .orderBy(cb.asc(root.get("name")));
14
15 // Stil 3: QueryDSL (typsicher, fließend)
16 List<User> users = new JPAQuery<>(em)
17     .select(QUser.user)
18     .from(QUser.user)
19     .where(QUser.user.active.isTrue())
20     .orderBy(QUser.user.name.asc())
21     .fetch();

```

Listing 2: Java – Query-Stilarten

2.2.3 C# – LINQ (Language Integrated Query)

C# bietet mit **LINQ** eine nativ in die Sprache integrierte Query-Syntax, die vom Compiler direkt in Methodenaufrufe übersetzt wird. Entity Framework Core implementiert

den IQueryable<T>-Provider für SQL-Generierung [5, 6]:

```
1 // Stil 1: LINQ Query Syntax (deklarativ, SQL-ähnlich)
2 var users = from u in context.Users
3             where u.Active == true
4             orderby u.Name
5             select u;
6
7 // Stil 2: LINQ Method Syntax (fliessend, Lambda)
8 var users = context.Users
9             .Where(u => u.Active == true)
10            .OrderBy(u => u.Name)
11            .ToList();
12
13 // Stil 3: LINQ mit anonymen Typen (Projektion)
14 var result = context.Users
15             .Where(u => u.Active)
16             .Select(u => new { u.Id, u.Name, u.Email });
```

Listing 3: C# LINQ – Query-Stilarten

2.3 Bestehende Ansätze und deren Limitierungen

Tabelle 1 fasst bestehende Ansätze zur Query-Abstraktion und deren Grenzen zusammen.

Tabelle 1: *Bestehende Query-Abstraktionsansätze und ihre Limitierungen*

Ansatz	Merkmale	Limitierungen
Hibernate (Java)	Mächtiges ORM, HQL, Criteria API	Nur Java; erzeugt oft ineffiziente SQL; kein sprachübergreifender Standard
Entity Framework (C#)	LINQ-Integration, Migrations	Nur .NET; SQL-Ausgabe variiert je nach EF-Version
SQLAlchemy (Python)	Flexibel, Core + ORM	Nur Python; verschiedene API-Generationen; inkonsistente SQL-Ausgabe
JOOQ (Java)	Typsicherer SQL-Builder	Nur Java; erfordert Code-Generierung; kommerziell für Enterprise
Exposed (Kotlin)	Kotlin DSL für SQL	Nur JVM/Kotlin; junges Ökosystem
Prisma (Node.js)	Typsicherer ORM für TypeScript	Nur Node.js/TypeScript; anderes Ökosystem
GraphQL-to-SQL	Graphenabfragen über SQL	Anderes Paradigma; nicht auf Standard-CRUD ausgerichtet

Kritische Feststellung: Kein bestehender Ansatz garantiert sprachübergreifende SQL-Äquivalenz. Die Lücke liegt nicht in der fehlenden Implementierung, sondern im Fehlen eines formalen Standards, der Äquivalenzanforderungen und Normalisierungsregeln verbindlich definiert.

2.4 Formale Grundbegriffe

Definition 2.2 (Query-Ausdruck). Ein *Query-Ausdruck* Q ist ein Term einer Operatoralgebra $\mathcal{A}$ über einer Datenbankschema-Instanz σ . Die *Semantik* von Q ist die Relation $\llbracket Q \rrbracket_\sigma$, die durch Auswertung von Q in σ entsteht.

Definition 2.3 (Semantische Äquivalenz). Zwei Query-Ausdrücke Q_1 und Q_2 heißen *semantisch äquivalent*, geschrieben $Q_1 \equiv Q_2$, wenn für alle Datenbankinstanzen σ gilt:

$$\llbracket Q_1 \rrbracket_\sigma = \llbracket Q_2 \rrbracket_\sigma$$

Definition 2.4 (UQTL-Konformität). Ein Übersetzungssystem T heißt *UQTL-konform*, wenn für alle äquivalenten Quellenanfragen $Q_{py} \equiv Q_{java} \equiv Q_{cs}$ gilt:

$$T(Q_{py}) = T(Q_{java}) = T(Q_{cs}) = S^*$$

wobei S^* das normalisierte SQL-Statement im UQTL-Normalformat ist.

Diese Definitionen bilden das formale Fundament des gesamten Standards.

3 Der Intermediate Query Tree (IQT)

3.1 Überblick und Architektur

Das Herzstück des UQTL-Standards ist der **Intermediate Query Tree (IQT)**: eine sprachunabhängige Baumstruktur, die jeden Query-Ausdruck aus Python, Java und C# einheitlich repräsentiert. Der IQT fungiert als *Pivot-Sprache* in der Übersetzungspipeline.

3.2 IQT-Knotentypen

Der IQT ist ein gerichteter azyklischer Baum (DAG) aus typisierten Knoten. Jeder Knoten entspricht einem relationalen oder logischen Operator.

Definition 3.1 (IQT-Knotentypen). Die Menge der zulässigen IQT-Knotentypen $\mathcal{N}$ ist definiert als:

$$\mathcal{N} = \{\text{SCAN, FILTER, PROJECT, JOIN, GROUP_AGG, SORT, LIMIT, WINDOW, SUBQUERY, SET_OP, WITH}\}$$

3.2.1 Knoten-Spezifikation im Detail

SCAN-Knoten Der SCAN-Knoten repräsentiert den Basistabellenzugriff:

```
1 SCAN {
2   table      : QualifiedName      -- Schema + Tabellenname
3   alias      : Identifier?        -- optionaler Alias
4   hint       : IndexHint?        -- optionaler Index-Hint (Erweiterung)
5 }
```

Listing 4: *IQT-Pseudonotation: SCAN*

FILTER-Knoten

```
1 FILTER {
2   source      : IQTNode           -- Eingabere relation
3   predicate   : Expr             -- boolescher Ausdruck
4   position    : {WHERE | HAVING}
5 }
6
7 Expr ::=
8   | CompareExpr { left: Expr, op: {EQ|NEQ|LT|GT|LTE|GTE}, right: Expr
9   | LogicExpr  { op: {AND|OR|NOT}, operands: Expr+ }
```

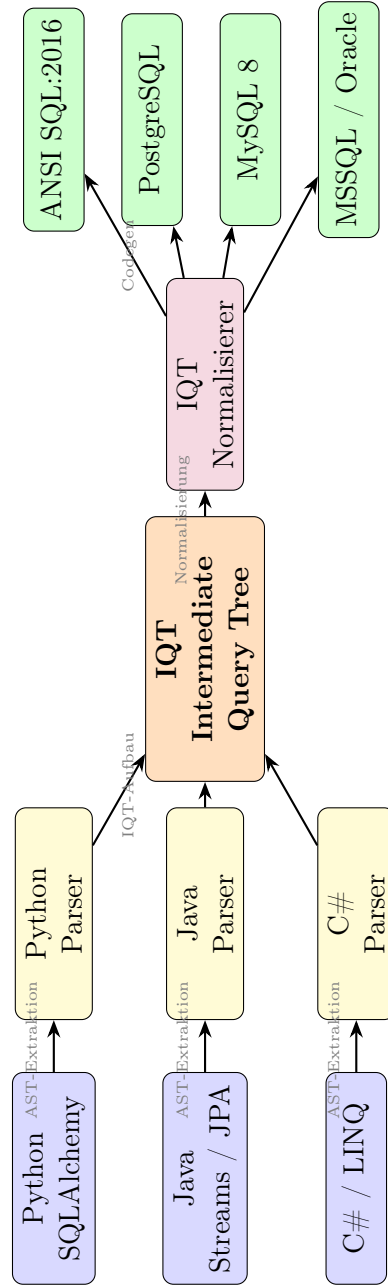


Abbildung 1: *UQTL-Übersetzungspipeline: von Quellsprachen über IQT zum SQL-Zieldialekt*

```

10 | InExpr      { value: Expr, list: Expr+ | SubQuery }
11 | BetweenExpr { value: Expr, lo: Expr, hi: Expr }
12 | LikeExpr   { value: Expr, pattern: StringLiteral, escape: Char? }
13 | NullExpr   { value: Expr, negated: Bool }
14 | ExistsExpr { subquery: SubQuery, negated: Bool }
15 | Literal    { type: DataType, value: Any }
16 | ColumnRef  { table: Identifier?, column: Identifier }
17 | FuncCall   { name: QualifiedName, args: Expr*, distinct: Bool }

```

Listing 5: *IQT-Pseudonotation: FILTER*

PROJECT-Knoten

```

1 PROJECT {
2   source      : IQTNode
3   columns     : ProjectionItem+
4   distinct    : Bool
5 }
6
7 ProjectionItem ::=
8   | Star      {}
9   | Expression { expr: Expr, alias: Identifier? }

```

Listing 6: *IQT-Pseudonotation: PROJECT*

JOIN-Knoten

```

1 JOIN {
2   left      : IQTNode
3   right     : IQTNode
4   type      : {INNER | LEFT_OUTER | RIGHT_OUTER | FULL_OUTER | CROSS}
5   condition : JoinCondition
6 }
7
8 JoinCondition ::=
9   | OnCondition   { predicate: Expr }
10  | UsingCondition { columns: Identifier+ }
11  | Natural       {}

```

Listing 7: *IQT-Pseudonotation: JOIN*

GROUP_AGG-Knoten

```

1 GROUP_AGG {
2   source      : IQTNode
3   groupBy     : Expr*           -- leer = Gesamt-Aggregation
4   aggregates  : AggItem+
5   having      : Expr?

```

```

6 }
7
8 AggItem ::= {
9   func   : {COUNT|SUM|AVG|MIN|MAX|STDDEV|VAR|STRING_AGG|...}
10  arg    : Expr           -- COUNT(*) => StarExpr
11  distinct: Bool
12  alias  : Identifier?
13 }

```

Listing 8: *IQT-Pseudonotation: GROUP_AGG*

SORT-Knoten

```

1 SORT {
2   source  : IQTNode
3   items   : SortItem+
4 }
5
6 SortItem ::= {
7   expr      : Expr
8   direction : {ASC | DESC}
9   nulls     : {NULLS_FIRST | NULLS_LAST | DEFAULT}
10 }

```

Listing 9: *IQT-Pseudonotation: SORT*

LIMIT-Knoten

```

1 LIMIT {
2   source : IQTNode
3   count  : Expr | Null    -- NULL = kein Limit
4   offset : Expr | Null    -- NULL = kein Offset
5 }

```

Listing 10: *IQT-Pseudonotation: LIMIT*

WINDOW-Knoten

```

1 WINDOW {
2   source      : IQTNode
3   definitions : WindowDef+
4 }
5
6 WindowDef ::= {
7   alias      : Identifier
8   partitionBy : Expr*
9   orderBy    : SortItem*
10  frame      : WindowFrame?

```

```

11  func          : WindowFunc
12  resultAlias   : Identifier
13  }
14
15  WindowFrame ::= {
16    unit   : {ROWS | RANGE}
17    start  : FrameBound
18    end    : FrameBound
19  }
20
21  FrameBound ::=
22    | UNBOUNDED_PRECEDING | CURRENT_ROW | UNBOUNDED_FOLLOWING
23    | Preceding { n: Expr } | Following { n: Expr }

```

Listing 11: *IQT-Pseudonotation: WINDOW*

3.3 Normalisierungsregeln des IQT

Um die eindeutige SQL-Ausgabe zu garantieren, werden IQT-Bäume vor der SQL-Generierung normalisiert. Die Normalisierung umfasst folgende Schritte:

Übersetzungsregel 3.1 (Prädikat-Normalisierung). Zusammengesetzte AND-Prädikate werden in einer kanonischen Reihenfolge sortiert (lexikografisch nach Spaltenname, dann Operator):

$$p_1 \wedge p_2 \wedge \dots \wedge p_n \rightarrow p_{\sigma(1)} \wedge p_{\sigma(2)} \wedge \dots \wedge p_{\sigma(n)}$$

mit σ als lexikografischer Permutation.

Übersetzungsregel 3.2 (Alias-Normalisierung). Generierte Tabellenalias folgen dem Schema $t_1, t_2, \dots$ in der Reihenfolge des Auftretens im Baum (Pre-Order-Traversal).

Übersetzungsregel 3.3 (Boolean-Vereinfachung). Redundante boolesche Ausdrücke werden eliminiert:

$$\begin{aligned}
p \wedge \text{TRUE} &\rightarrow p \\
p \vee \text{FALSE} &\rightarrow p \\
\neg \neg p &\rightarrow p \\
p \wedge p &\rightarrow p
\end{aligned}$$

Übersetzungsregel 3.4 (SELECT-Reihenfolge). Explizit genannte Projektionsspalten werden in der Reihenfolge ihrer Deklaration ausgegeben. **SELECT *** wird nur bei echten Stern-Projektionen verwendet; implizite Stern-Projektionen werden beibehalten.

Übersetzungsregel 3.5 (JOIN-Kanonisierung). Mehrfach-Joins werden left-assoziativ normiert. **CROSS JOIN** ohne **ON**-Klausel wird nur verwendet, wenn explizit kein Prädikat vorliegt; sonst wird **INNER JOIN ... ON** bevorzugt.

4 Übersetzungsregeln und Operatoren

Dieses Kapitel definiert die vollständigen Übersetzungsregeln von den Quellsprachen Python, Java und C# in den IQT sowie vom IQT nach ANSI SQL. Die Regeln werden als Transformationstabellen und formale Schreibweisen dargestellt.

4.1 Operator 1: Filterung (WHERE / HAVING)

SQL-Generierungsregel für FILTER:

$$\begin{aligned}\text{SQL}(\text{FILTER}(S, p, \text{WHERE})) &= \text{SQL}(S) + \text{ WHERE } + \text{SQL}(p) \\ \text{SQL}(\text{FILTER}(S, p, \text{HAVING})) &= \text{SQL}(S) + \text{ HAVING } + \text{SQL}(p)\end{aligned}$$

4.2 Operator 2: Projektion (SELECT)

4.3 Operator 3: Sortierung (ORDER BY)

SQL-Generierungsregel:

$$\begin{aligned}\text{SQL}(\text{SORT}(S, [s_1, \dots, s_n])) &= \text{SQL}(S) + \text{ ORDER BY } + \text{SQL}(s_1) + , + \dots + , + \text{SQL}(s_n) \\ \text{SQL}(\text{SortItem}(e, \text{ASC}, \text{DEFAULT})) &= \text{SQL}(e) + \text{ ASC }\end{aligned}$$

4.4 Operator 4: Begrenzung (LIMIT / OFFSET)

4.5 Operator 5: Aggregation (GROUP BY)

4.6 Operator 6: Joins

Übersetzungsregel 4.1 (JOIN-Übersetzung). Für einen INNER JOIN zwischen Relation L (Alias t_1) und R (Alias t_2) mit Prädikat p ergibt sich:

$$\text{SQL}(\text{JOIN}(L, R, \text{INNER}, p)) = \text{SQL}(L) + \text{ INNER JOIN } + \text{SQL}(R) + \text{ ON } + \text{SQL}(p)$$

4.7 Operator 7: Fensterfunktionen (OVER)

Fensterfunktionen (Window Functions, ANSI SQL:2003+) werden in UQTL ab Konformitätsstufe CL-3 unterstützt.

Tabelle 2: Übersetzungstabelle: Filteroperatoren

IQT-Prädikat	Python	Java	C#
$EQ(a, v)$	<code>Model.col == v</code>	<code>cb.equal(r.get("col"), v)</code>	<code>x.Col == v</code>
$NEQ(a, v)$	<code>Model.col != v</code>	<code>cb.notEqual(...)</code>	<code>x.Col != v</code>
$GT(a, v)$	<code>Model.col > v</code>	<code>cb.gt(...)</code>	<code>x.Col > v</code>
$GTE(a, v)$	<code>Model.col >= v</code>	<code>cb.ge(...)</code>	<code>x.Col >= v</code>
$IN(a, list)$	<code>Model.col.in_(list)</code>	<code>root.get("col").in(list)</code>	<code>list.Contains(x.Col)</code>
BETWEEN	<code>col.between(a,b)</code>	<code>cb.between(r.get("col"), a,b)</code>	<code>x.Col >= a && x.Col <= b</code>
$LIKE(a, p)$	<code>Model.col.like(p)</code>	<code>cb.like(r.get("col"), p)</code>	<code>EF.Functions.Like(x.Col,p)</code>
IS_NULL	<code>Model.col == None</code>	<code>cb.isNull(r.get("col"))</code>	<code>x.Col == null</code>
AND	<code>& oder and_()</code>	<code>cb.and(...)</code>	<code>&&</code>
OR	<code> oder or_()</code>	<code>cb.or(...)</code>	<code> </code>

Tabelle 3: *Übersetzungstabelle: Projektionsoperatoren*

Operation	Python	Java	C#
Alle Spalten	<code>select (Model)</code>	<code>cq.select (root)</code>	<code>.Select (x => x)</code>
Einzelne Spalten	<code>select (M.a, M.b)</code>	<code>cb.construct (DT0, r.get ("ä"), r.get ("b"))</code>	<code>.Select (x => new {x.A, x.B})</code>
Ausdruck	<code>select (func.upper (M.name))</code>	<code>cb.upper (root.get ("name"))</code>	<code>.Select (x => x.Name.ToUpper ())</code>
Distinct	<code>select (M.col).distinct ()</code>	<code>cq.distinct (true)</code>	<code>.Select (x => x.Col).Distinct ()</code>

Tabelle 4: *Übersetzungstabelle: Sortieroperatoren*

IQT-Operation	Python	Java	C#
ASC	<code>order_by(M.col)</code>	<code>cb.asc(r.get("col"))</code>	<code>OrderBy(x => x.Col)</code>
DESC	<code>order_by(desc(M.col))</code>	<code>cb.desc(r.get("col"))</code>	<code>OrderByDescending(x => x.Col)</code>
Multi-Sort	<code>order_by(M.a, desc(M.b))</code>	<code>.orderBy(cb.asc(...), cb.desc(...))</code>	<code>.OrderBy(x => x.A).ThenByDescending(x => x.B)</code>
NULLS FIRST	<code>nullsfirst(M.col)</code>	<code>cb.asc(r.get("col")).nullsfirst()</code>	<code>EF nicht nativ</code>

Tabelle 5: Übersetzungstabelle: *LIMIT* / *OFFSET* je Datenbankdialekt

Sprache / Dialekt	Quellausdruck	SQL-Ausgabe
Python (SA)	<code>.limit(10).offset(20)</code>	<code>LIMIT 10 OFFSET 20</code>
Java (JPA)	<code>.setMaxResults(10).setFirstResult(20)</code>	<code>LIMIT 10 OFFSET 20</code>
C# (LINQ)	<code>.Take(10).Skip(20)</code>	<code>LIMIT 10 OFFSET 20</code>
<i>Dialektspezifische Ausgabe (Erweiterungsmodus):</i>		
MSSQL	– (wie oben)	<code>OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY</code>
Oracle <12c	– (wie oben)	<code>WHERE ROWNUM BETWEEN 21 AND 30 (umgeschrieben)</code>

Tabelle 6: Übersetzungstabelle: Aggregationsfunktionen

IQT-Funktion	Python	Java	C#
COUNT(*)	func.count('*')	cb.count(root)	.Count()
COUNT(col)	func.count(M.col)	cb.count(r.get("col"))	.Count(x => x.Col != null)
SUM(col)	func.sum(M.col)	cb.sum(r.get("col"))	.Sum(x => x.Col)
AVG(col)	func.avg(M.col)	cb.avg(r.get("col"))	.Average(x => x.Col)
MIN(col)	func.min(M.col)	cb.min(r.get("col"))	.Min(x => x.Col)
MAX(col)	func.max(M.col)	cb.max(r.get("col"))	.Max(x => x.Col)
GROUP BY	group_by(M.dep)	cq.groupBy(r.get("dep"))	.GroupBy(x => x.Dep)
HAVING	having(func.count('*') > 5)	cq.having(cb.gt(cb.count(r), 5))	.Where(g => g.Count() > 5)

Tabelle 7: Übersetzungstabelle: Join-Typen

Join-Typ	Python	Java	C#
INNER JOIN	<code>join(Order, User.id == Order.user_id)</code>	<code>root.join(orders")</code>	<code>join in context.Orders on u.Id equals o.UserId</code>
LEFT OUTER	<code>outerjoin(Order, ...)</code>	<code>root.join(..., JoinType.LEFT)</code>	<code>join o in ... into grp from o in grp.DefaultIfEmpty()</code>
RIGHT OUTER	(SA kein nativer Support)	<code>JoinType.RIGHT</code>	(symmetrisch zu LEFT)
CROSS JOIN	<code>select_from(A, B)</code>	<code>cq.from(A.class); cq.from(B.class)</code>	<code>from a in As from b in Bs select ...</code>

Tabelle 8: *Übersetzungstabelle: Fensterfunktionen*

SQL-Funktion	Python	Java	C#
ROW_NUMBER()	<code>func.row_number().over(...)</code>	<code>cb.rowNumber().over(...)</code>	<code>EF.Functions.RowNumber(...)</code>
RANK()	<code>func.rank().over(...)</code>	<code>cb.rank().over(...)</code>	<code>EF.Functions.Rank(...)</code>
DENSE_RANK()	<code>func.dense_rank().over(...)</code>	<code>cb.denseRank().over(...)</code>	<code>EF.Functions.DenseRank(...)</code>
LAG(col, n)	<code>func.lag(M.col, n).over(...)</code>	<code>cb.lag(r.get("col"), n)</code>	<code>EF.Functions.Lag(x.Col, n)</code>
LEAD(col, n)	<code>func.lead(M.col, n).over(...)</code>	<code>cb.lead(r.get("col"), n)</code>	<code>EF.Functions.Lead(x.Col, n)</code>
SUM() OVER	<code>func.sum(M.col).over(...)</code>	<code>cb.sum(r.get("col")).over(...)</code>	<code>.Sum(x.Col).Over(...)</code>

Tabelle 9: Übersetzungstabelle: Unterabfragen

Typ	Python	Java	C#
Scalar Subquery	subquery()	cb.subquery(Long.class)	context.Baz.Where(...).Select(x => x.Val).FirstOrDefault()
EXISTS	exists(subq)	cb.exists(subq)	.Any(cond)
IN (Subquery)	M.col.in_(subq)	root.get("col").in(subq)	list.Contains(x.Col) (mit AsQueryable)
CTE (WITH)	cte = subq.cte("name")	(JPA kein nativer CTE)	(EF Core ≥ 7 : FromSqlRaw)

4.8 Operator 8: Unterabfragen und CTEs

4.9 Operator 9: Mengenoperationen

Tabelle 10: Übersetzungstabelle: Mengenoperationen

IQT	Python	Java	C#
UNION	<code>q1.union(q2)</code>	(via CriteriaQuery)	<code>q1.Union(q2)</code>
UNION ALL	<code>q1.union_all(q2)</code>	(JPA: JPQL UNION)	<code>q1.Concat(q2)</code>
INTERSECT	<code>q1.intersect(q2)</code>	(selten nativ)	<code>q1.Intersect(q2)</code>
EXCEPT	<code>q1.except_(q2)</code>	(selten nativ)	<code>q1.Except(q2)</code>

5 Praxisbeispiele

Dieses Kapitel demonstriert den UQTL-Standard anhand vollständiger, realistischer Datenbankszenarien. Für jedes Beispiel wird zunächst das Datenbankschema beschrieben, dann die Abfrage in allen drei Quellsprachen formuliert, und schließlich das vom Standard definierte normalisierte SQL-Ergebnis angegeben.

5.1 Datenmodell für die Beispiele

Alle Beispiele basieren auf folgendem vereinfachten E-Commerce-Schema:

```
1 CREATE TABLE customers (  
2     id            INTEGER PRIMARY KEY,  
3     name          VARCHAR(200) NOT NULL,  
4     email         VARCHAR(200) UNIQUE,  
5     country       VARCHAR(50),  
6     active        BOOLEAN NOT NULL DEFAULT TRUE,  
7     created_at    TIMESTAMP NOT NULL,  
8     tier           VARCHAR(20) -- 'bronze', 'silver', 'gold', 'platinum'  
9 );  
10  
11 CREATE TABLE orders (  
12     id            INTEGER PRIMARY KEY,  
13     customer_id   INTEGER REFERENCES customers(id),  
14     order_date    DATE NOT NULL,  
15     status        VARCHAR(30), --  
16                 'pending', 'confirmed', 'shipped', 'delivered'  
17     total         DECIMAL(12,2) NOT NULL  
18 );  
19  
20 CREATE TABLE order_items (  
21     id            INTEGER PRIMARY KEY,  
22     order_id      INTEGER REFERENCES orders(id),  
23     product_id    INTEGER REFERENCES products(id),  
24     quantity      INTEGER NOT NULL,  
25     unit_price    DECIMAL(10,2) NOT NULL  
26 );  
27  
28 CREATE TABLE products (  
29     id            INTEGER PRIMARY KEY,  
30     name          VARCHAR(200) NOT NULL,  
31     category      VARCHAR(100),  
32     price         DECIMAL(10,2) NOT NULL,  
33     stock         INTEGER NOT NULL DEFAULT 0  
34 );
```

Listing 12: *Datenbankschema für Beispiele*

5.2 Beispiel 1: Einfache Filterung und Projektion

Aufgabe: Alle aktiven Kunden aus Deutschland, nur Name und E-Mail, alphabetisch sortiert.

```
1 from sqlalchemy import select
2 from models import Customer
3
4 stmt = (
5     select(Customer.name, Customer.email)
6     .where(Customer.active == True)
7     .where(Customer.country == "DE")
8     .order_by(Customer.name.asc())
9 )
10 results = session.execute(stmt).all()
```

Listing 13: *Beispiel 1 – Python (SQLAlchemy 2.0)*

```
1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
3 Root<Customer> c = cq.from(Customer.class);
4
5 cq.multiselect(c.get("name"), c.get("email"))
6     .where(
7         cb.and(
8             cb.isTrue(c.get("active")),
9             cb.equal(c.get("country"), "DE")
10        )
11    )
12    .orderBy(cb.asc(c.get("name")));
13
14 List<Object[]> results = em.createQuery(cq).getResultList();
```

Listing 14: *Beispiel 1 – Java (JPA Criteria API)*

```
1 // Query Syntax
2 var results = from c in context.Customers
3     where c.Active == true && c.Country == "DE"
4     orderby c.Name ascending
5     select new { c.Name, c.Email };
6
7 // Equivalent Method Syntax
8 var results = context.Customers
9     .Where(c => c.Active == true && c.Country == "DE")
10    .OrderBy(c => c.Name)
11    .Select(c => new { c.Name, c.Email });
```

Listing 15: *Beispiel 1 – C# (EF Core / LINQ)*

UQTL-Normiertes SQL (identisch für alle drei Quellsprachen):

```
SELECT t1.name, t1.email
FROM customers AS t1
WHERE t1.active = TRUE
      AND t1.country = 'DE'
ORDER BY t1.name ASC
```

5.3 Beispiel 2: Aggregation mit GROUP BY und HAVING

Aufgabe: Anzahl bestätigter Bestellungen und Gesamtumsatz pro Kunde, nur Kunden mit mehr als 3 Bestellungen, absteigend nach Umsatz.

```
1 from sqlalchemy import select, func
2
3 stmt = (
4     select(
5         Order.customer_id,
6         func.count(Order.id).label("order_count"),
7         func.sum(Order.total).label("total_revenue")
8     )
9     .where(Order.status == "confirmed")
10    .group_by(Order.customer_id)
11    .having(func.count(Order.id) > 3)
12    .order_by(func.sum(Order.total).desc())
13 )
```

Listing 16: *Beispiel 2 – Python*

```
1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
3 Root<Order> o = cq.from(Order.class);
4
5 Expression<Long> cnt = cb.count(o.get("id"));
6 Expression<Double> rev = cb.sum(o.<Double>get("total"));
7
8 cq.multiselect(o.get("customerId"), cnt, rev)
9   .where(cb.equal(o.get("status"), "confirmed"))
10  .groupBy(o.get("customerId"))
11  .having(cb.gt(cnt, 3L))
12  .orderBy(cb.desc(rev));
```

Listing 17: *Beispiel 2 – Java*

```
1 var results = context.Orders
2   .Where(o => o.Status == "confirmed")
```

```

3      .GroupBy(o => o.CustomerId)
4      .Where(g => g.Count() > 3)
5      .OrderByDescending(g => g.Sum(o => o.Total))
6      .Select(g => new {
7          CustomerId    = g.Key,
8          OrderCount    = g.Count(),
9          TotalRevenue  = g.Sum(o => o.Total)
10     });

```

Listing 18: *Beispiel 2 – C#*

UQTL-Normiertes SQL:

```

SELECT t1.customer_id,
       COUNT(t1.id)    AS order_count,
       SUM(t1.total)   AS total_revenue
FROM   orders AS t1
WHERE  t1.status = 'confirmed'
GROUP BY t1.customer_id
HAVING COUNT(t1.id) > 3
ORDER BY SUM(t1.total) DESC

```

5.4 Beispiel 3: INNER JOIN mit mehreren Bedingungen

Aufgabe: Name und E-Mail der Kunden, die mindestens eine gelieferte Bestellung im Wert über 500 EUR hatten, im Jahr 2024.

```

1 from sqlalchemy import select, extract
2
3 stmt = (
4     select(Customer.name, Customer.email)
5     .join(Order, Customer.id == Order.customer_id)
6     .where(Order.status == "delivered")
7     .where(Order.total > 500)
8     .where(extract("year", Order.order_date) == 2024)
9     .distinct()
10 )

```

Listing 19: *Beispiel 3 – Python (JOIN)*

```

1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
3 Root<Customer> c = cq.from(Customer.class);
4 Join<Customer, Order> o = c.join("orders", JoinType.INNER);
5
6 cq.multiselect(c.get("name"), c.get("email"))
7     .where(cb.and(

```

```

8      cb.equal(o.get("status"), "delivered"),
9      cb.gt(o.<BigDecimal>get("total"), new BigDecimal("500")),
10     cb.equal(
11         cb.function("YEAR", Integer.class, o.get("orderDate")),
12         2024
13     )
14 ))
15 .distinct(true);

```

Listing 20: *Beispiel 3 – Java (JOIN)*

```

1 var results = (from c in context.Customers
2               join o in context.Orders
3               on c.Id equals o.CustomerId
4               where o.Status == "delivered"
5                   && o.Total > 500m
6                   && o.OrderDate.Year == 2024
7               select new { c.Name, c.Email })
8 .Distinct();

```

Listing 21: *Beispiel 3 – C# (JOIN)*

UQTL-Normiertes SQL:

```

SELECT DISTINCT t1.name, t1.email
FROM customers AS t1
INNER JOIN orders AS t2
  ON t1.id = t2.customer_id
WHERE t2.status = 'delivered'
  AND t2.total > 500
  AND EXTRACT(YEAR FROM t2.order_date) = 2024

```

5.5 Beispiel 4: LEFT OUTER JOIN und NULL-Behandlung

Aufgabe: Alle Kunden mit Anzahl ihrer Bestellungen (auch solche ohne Bestellungen).

```

1 from sqlalchemy import select, func, outerjoin
2
3 stmt = (
4     select(
5         Customer.id,
6         Customer.name,
7         func.count(Order.id).label("order_count")
8     )
9     .outerjoin(Order, Customer.id == Order.customer_id)
10    .group_by(Customer.id, Customer.name)
11    .order_by(Customer.name)

```

12)

Listing 22: *Beispiel 4 – Python (LEFT JOIN)*

```
1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
3 Root<Customer> c = cq.from(Customer.class);
4 Join<Customer, Order> o = c.join("orders", JoinType.LEFT);
5
6 cq.multiselect(c.get("id"), c.get("name"), cb.count(o.get("id")))
7   .groupBy(c.get("id"), c.get("name"))
8   .orderBy(cb.asc(c.get("name")));
```

Listing 23: *Beispiel 4 – Java (LEFT JOIN)*

```
1 var results = from c in context.Customers
2               join o in context.Orders
3                 on c.Id equals o.CustomerId into orders
4               from o in orders.DefaultIfEmpty()
5               group o by new { c.Id, c.Name } into g
6               orderby g.Key.Name
7               select new {
8                   g.Key.Id,
9                   g.Key.Name,
10                  OrderCount = g.Count(o => o != null)
11               };
```

Listing 24: *Beispiel 4 – C# (LEFT JOIN)*

UQTL-Normiertes SQL:

```
SELECT t1.id, t1.name, COUNT(t2.id) AS order_count
FROM customers AS t1
LEFT OUTER JOIN orders AS t2
  ON t1.id = t2.customer_id
GROUP BY t1.id, t1.name
ORDER BY t1.name ASC
```

5.6 Beispiel 5: Fensterfunktionen (OVER PARTITION BY)

Aufgabe: Für jede Bestellung: Rang innerhalb des Kunden nach Bestellwert (teuerste = Rang 1), laufende Summe der Bestellwerte pro Kunde.

```
1 from sqlalchemy import select, func
2 from sqlalchemy import over
3
4 rnk = func.rank().over(
```



```

5     partition_by=Order.customer_id,
6     order_by=Order.total.desc()
7 ).label("rnk")
8
9 run_total = func.sum(Order.total).over(
10     partition_by=Order.customer_id,
11     order_by=Order.order_date.asc(),
12     rows=(None, 0) # UNBOUNDED PRECEDING to CURRENT ROW
13 ).label("running_total")
14
15 stmt = select(
16     Order.id,
17     Order.customer_id,
18     Order.total,
19     rnk,
20     run_total
21 ).order_by(Order.customer_id, rnk)

```

Listing 25: *Beispiel 5 – Python (Window Functions)*

```

1 // Java JPA hat keine native Window Function API.
2 // Blaze-Persistence oder native Queries werden benoetigt:
3 String jpql = """
4     SELECT o.id, o.customerId, o.total,
5           RANK() OVER (PARTITION BY o.customerId ORDER BY o.total
6                        DESC),
7           SUM(o.total) OVER (
8               PARTITION BY o.customerId
9               ORDER BY o.orderDate
10              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
11          )
12 FROM Order o
13 ORDER BY o.customerId, 4
14 """;
15 List<Object[]> results = em.createQuery(jpql).getResultList();

```

Listing 26: *Beispiel 5 – Java (Window via JPQL / Blaze-Persistence)*

```

1 // EF Core 8+ mit Window Function Unterstuetzung:
2 var results = context.Orders
3     .Select(o => new {
4         o.Id,
5         o.CustomerId,
6         o.Total,
7         Rank = EF.Functions.Rank()
8             .Over()
9             .PartitionBy(o.CustomerId)

```

```

10         .OrderByDescending(o.Total),
11         RunningTotal = EF.Functions.Sum(o.Total)
12         .Over()
13         .PartitionBy(o.CustomerId)
14         .OrderBy(o.OrderDate)
15         .RowsBetween(
16             EF.Functions.RowsOrRangeBoundary.UnboundedPreceding,
17             EF.Functions.RowsOrRangeBoundary.CurrentRow)
18     })
19     .OrderBy(x => x.CustomerId)
20     .ThenBy(x => x.Rank);

```

Listing 27: *Beispiel 5 – C# (EF Core Window Functions)*

UQTL-Normiertes SQL:

```

SELECT t1.id,
       t1.customer_id,
       t1.total,
       RANK() OVER (
           PARTITION BY t1.customer_id
           ORDER BY t1.total DESC
       ) AS rnk,
       SUM(t1.total) OVER (
           PARTITION BY t1.customer_id
           ORDER BY t1.order_date ASC
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM orders AS t1
ORDER BY t1.customer_id ASC, rnk ASC

```

5.7 Beispiel 6: Unterabfragen (Subqueries)

Aufgabe: Kunden, deren letzter Bestellwert über dem Durchschnitt aller Bestellwerte liegt.

```

1 from sqlalchemy import select, func
2
3 # Unterabfrage: Durchschnittlicher Bestellwert
4 avg_subq = select(func.avg(Order.total)).scalar_subquery()
5
6 # Unterabfrage: Letzte Bestellung je Kunde
7 last_order = (
8     select(Order.customer_id,
9            func.max(Order.order_date).label("last_date"))
10    .group_by(Order.customer_id)

```

```

10     .subquery()
11 )
12
13 # Hauptabfrage
14 stmt = (
15     select(Customer.id, Customer.name)
16     .join(last_order, Customer.id == last_order.c.customer_id)
17     .join(Order, (Order.customer_id == Customer.id) &
18               (Order.order_date == last_order.c.last_date))
19     .where(Order.total > avg_subq)
20     .order_by(Customer.name)
21 )

```

Listing 28: *Beispiel 6 – Python (Subquery)*

```

1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
3 Root<Customer> c = cq.from(Customer.class);
4 Join<Customer, Order> o = c.join("orders", JoinType.INNER);
5
6 // Scalar Subquery: Durchschnitt
7 Subquery<Double> avgSub = cq.subquery(Double.class);
8 Root<Order> so = avgSub.from(Order.class);
9 avgSub.select(cb.avg(so.<Double>get("total")));
10
11 cq.multiselect(c.get("id"), c.get("name"))
12   .where(cb.and(
13       cb.gt(o.<Double>get("total"), avgSub),
14       // letzte Bestellung via correlated subquery ...
15       cb.equal(o.get("orderDate"),
16               cb.greatest(o.<java.time.LocalDate>get("orderDate")))
17   ))
18   .orderBy(cb.asc(c.get("name")));

```

Listing 29: *Beispiel 6 – Java (Subquery)*

```

1 var avgTotal = context.Orders.Average(o => o.Total);
2
3 var lastOrders = context.Orders
4   .GroupBy(o => o.CustomerId)
5   .Select(g => new {
6       CustomerId = g.Key,
7       LastDate   = g.Max(o => o.OrderDate)
8   });
9
10 var results = from c in context.Customers
11               join lo in lastOrders on c.Id equals lo.CustomerId

```

```

12         join o in context.Orders
13         on new { c.Id, lo.LastDate }
14         equals new { Id = o.CustomerId, LastDate =
15             o.OrderDate }
16     where o.Total > avgTotal
17     orderby c.Name
18     select new { c.Id, c.Name };

```

Listing 30: *Beispiel 6 – C# (Subquery via LINQ)*

UQTL-Normiertes SQL:

```

SELECT t1.id, t1.name
FROM customers AS t1
INNER JOIN orders AS t2
    ON t1.id = t2.customer_id
WHERE t2.order_date = (
    SELECT MAX(t3.order_date)
    FROM orders AS t3
    WHERE t3.customer_id = t1.id
)
AND t2.total > (
    SELECT AVG(t4.total)
    FROM orders AS t4
)
ORDER BY t1.name ASC

```

5.8 Beispiel 7: Komplexer Multi-Join mit Aggregation

Aufgabe: Top-5-Produkte nach Umsatz (Menge × Einzelpreis), nur Produkte der Kategorie „Elektronik“, mit Anzahl unterschiedlicher Käufer.

```

1 from sqlalchemy import select, func
2
3 stmt = (
4     select(
5         Product.id,
6         Product.name,
7         func.sum(OrderItem.quantity *
8             OrderItem.unit_price).label("revenue"),
9         func.count(Order.customer_id.distinct()).label("unique_buyers")
10    )
11    .join(OrderItem, Product.id == OrderItem.product_id)
12    .join(Order, OrderItem.order_id == Order.id)
13    .where(Product.category == "Elektronik")
14    .group_by(Product.id, Product.name)

```

```

14     .order_by(func.sum(
15         OrderItem.quantity * OrderItem.unit_price).desc())
16     .limit(5)
17 )

```

Listing 31: *Beispiel 7 – Python (Multi-Join + Aggregation)*

```

1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
3 Root<Product> p = cq.from(Product.class);
4 Join<Product, OrderItem> oi = p.join("orderItems", JoinType.INNER);
5 Join<OrderItem, Order> o = oi.join("order", JoinType.INNER);
6
7 Expression<BigDecimal> revenue =
8     cb.sum(cb.prod(oi.<Integer>get("quantity"),
9         oi.<BigDecimal>get("unitPrice"))));
10 Expression<Long> buyers =
11     cb.countDistinct(o.get("customerId"));
12
13 cq.multiselect(p.get("id"), p.get("name"), revenue, buyers)
14     .where(cb.equal(p.get("category"), "Elektronik"))
15     .groupBy(p.get("id"), p.get("name"))
16     .orderBy(cb.desc(revenue));
17
18 List<Object[]> result = em.createQuery(cq)
19     .setMaxResults(5)
20     .getResultList();

```

Listing 32: *Beispiel 7 – Java (Multi-Join + Aggregation)*

```

1 var results = context.Products
2     .Where(p => p.Category == "Elektronik")
3     .Join(context.OrderItems,
4         p => p.Id,
5         oi => oi.ProductId,
6         (p, oi) => new { p, oi })
7     .Join(context.Orders,
8         x => x.oi.OrderId,
9         o => o.Id,
10        (x, o) => new { x.p, x.oi, o })
11     .GroupBy(x => new { x.p.Id, x.p.Name })
12     .Select(g => new {
13         g.Key.Id,
14         g.Key.Name,
15         Revenue = g.Sum(x => x.oi.Quantity * x.oi.UnitPrice),
16         UniqueBuyers = g.Select(x =>
17             x.o.CustomerId).Distinct().Count()
18     });

```

```

17     })
18     .OrderByDescending(x => x.Revenue)
19     .Take(5);

```

Listing 33: *Beispiel 7 – C# (Multi-Join + Aggregation)*

UQTL-Normiertes SQL:

```

SELECT t1.id,
       t1.name,
       SUM(t2.quantity * t2.unit_price) AS revenue,
       COUNT(DISTINCT t3.customer_id) AS unique_buyers
FROM products AS t1
INNER JOIN order_items AS t2
  ON t1.id = t2.product_id
INNER JOIN orders AS t3
  ON t2.order_id = t3.id
WHERE t1.category = 'Elektronik'
GROUP BY t1.id, t1.name
ORDER BY SUM(t2.quantity * t2.unit_price) DESC
LIMIT 5

```

5.9 Beispiel 8: EXISTS-Unterabfrage

Aufgabe: Alle Kunden, die mindestens eine Bestellung mit dem Status „shipped“ aber noch keine mit dem Status „delivered“ haben.

```

1 from sqlalchemy import select, exists
2
3 has_shipped = exists(
4     select(Order.id)
5     .where(Order.customer_id == Customer.id)
6     .where(Order.status == "shipped")
7 )
8 has_delivered = exists(
9     select(Order.id)
10    .where(Order.customer_id == Customer.id)
11    .where(Order.status == "delivered")
12 )
13
14 stmt = (
15     select(Customer)
16     .where(has_shipped)
17     .where(~has_delivered)
18     .order_by(Customer.name)
19 )

```

Listing 34: Beispiel 8 – Python (EXISTS)

```
1 CriteriaBuilder cb = em.getCriteriaBuilder();
2 CriteriaQuery<Customer> cq = cb.createQuery(Customer.class);
3 Root<Customer> c = cq.from(Customer.class);
4
5 Subquery<Integer> shipped = cq.subquery(Integer.class);
6 Root<Order> os = shipped.from(Order.class);
7 shipped.select(cb.literal(1))
8     .where(cb.and(
9         cb.equal(os.get("customerId"), c.get("id")),
10        cb.equal(os.get("status"), "shipped")
11    ));
12
13 Subquery<Integer> delivered = cq.subquery(Integer.class);
14 Root<Order> od = delivered.from(Order.class);
15 delivered.select(cb.literal(1))
16     .where(cb.and(
17         cb.equal(od.get("customerId"), c.get("id")),
18         cb.equal(od.get("status"), "delivered")
19    ));
20
21 cq.select(c)
22     .where(cb.and(
23         cb.exists(shipped),
24         cb.not(cb.exists(delivered))
25     ))
26     .orderBy(cb.asc(c.get("name")));
```

Listing 35: Beispiel 8 – Java (EXISTS)

```
1 var results = context.Customers
2     .Where(c =>
3         context.Orders.Any(o => o.CustomerId == c.Id
4             && o.Status == "shipped") &&
5         !context.Orders.Any(o => o.CustomerId == c.Id
6             && o.Status == "delivered"))
7     .OrderBy(c => c.Name);
```

Listing 36: Beispiel 8 – C# (EXISTS via Any)

UQTL-Normiertes SQL:

```
SELECT t1.id, t1.name, t1.email, t1.country,
       t1.active, t1.created_at, t1.tier
```

```

FROM customers AS t1
WHERE EXISTS (
    SELECT 1
    FROM orders AS t2
    WHERE t2.customer_id = t1.id
        AND t2.status = 'shipped'
)
AND NOT EXISTS (
    SELECT 1
    FROM orders AS t3
    WHERE t3.customer_id = t1.id
        AND t3.status = 'delivered'
)
ORDER BY t1.name ASC

```

5.10 Beispiel 9: Common Table Expressions (WITH)

Aufgabe: Kunden mit überdurchschnittlichem Umsatz (Top-Tier-Analyse), unter Verwendung von CTEs für Lesbarkeit.

```

1 from sqlalchemy import select, func
2
3 # CTE 1: Gesamtumsatz je Kunde
4 customer_revenue = (
5     select(
6         Order.customer_id,
7         func.sum(Order.total).label("total_revenue")
8     )
9     .group_by(Order.customer_id)
10    .cte("customer_revenue")
11 )
12
13 # CTE 2: Durchschnittsumsatz
14 avg_revenue = (
15     select(func.avg(customer_revenue.c.total_revenue).label("avg_rev"))
16     .cte("avg_revenue")
17 )
18
19 # Hauptabfrage
20 stmt = (
21     select(Customer.name, Customer.email,
22            customer_revenue.c.total_revenue)
23     .join(customer_revenue,
24           Customer.id == customer_revenue.c.customer_id)
25     .where(customer_revenue.c.total_revenue >
26            select(avg_revenue.c.avg_rev).scalar_subquery())

```



```

27     .order_by(customer_revenue.c.total_revenue.desc())
28 )

```

Listing 37: *Beispiel 9 – Python (CTE)*

```

1  // EF Core: CTEs werden als verkettete LINQ-Ausdrücke modelliert
2  var customerRevenue = context.Orders
3      .GroupBy(o => o.CustomerId)
4      .Select(g => new {
5          CustomerId    = g.Key,
6          TotalRevenue  = g.Sum(o => o.Total)
7      });
8
9  var avgRevenue = customerRevenue.Average(x => x.TotalRevenue);
10
11 var results = context.Customers
12     .Join(customerRevenue,
13         c => c.Id,
14         cr => cr.CustomerId,
15         (c, cr) => new { c, cr })
16     .Where(x => x.cr.TotalRevenue > avgRevenue)
17     .OrderByDescending(x => x.cr.TotalRevenue)
18     .Select(x => new {
19         x.c.Name,
20         x.c.Email,
21         x.cr.TotalRevenue
22     });

```

Listing 38: *Beispiel 9 – C# (CTE-Äquivalent)*

UQTL-Normiertes SQL (mit CTE):

```

WITH customer_revenue AS (
    SELECT t2.customer_id,
           SUM(t2.total) AS total_revenue
    FROM orders AS t2
    GROUP BY t2.customer_id
),
avg_revenue AS (
    SELECT AVG(cr.total_revenue) AS avg_rev
    FROM customer_revenue AS cr
)
SELECT t1.name, t1.email, cr.total_revenue
FROM customers AS t1
INNER JOIN customer_revenue AS cr
    ON t1.id = cr.customer_id
WHERE cr.total_revenue > (SELECT avg_rev FROM avg_revenue)

```

```
ORDER BY cr.total_revenue DESC
```

5.11 Beispiel 10: Vollständige Geschäftslogik-Abfrage

Aufgabe: Monatlicher Umsatzbericht für das Jahr 2024: Monat, Gesamtumsatz, Anzahl Bestellungen, Anzahl eindeutiger Kunden, Vergleich zum Vormonat (via LAG-Fensterfunktion).

```
1 from sqlalchemy import select, func, extract
2
3 monthly = (
4     select(
5         extract("year", Order.order_date).label("year"),
6         extract("month", Order.order_date).label("month"),
7         func.sum(Order.total).label("revenue"),
8         func.count(Order.id).label("order_count"),
9         func.count(Order.customer_id.distinct()).label("unique_customers")
10    )
11    .where(extract("year", Order.order_date) == 2024)
12    .where(Order.status != "pending")
13    .group_by(
14        extract("year", Order.order_date),
15        extract("month", Order.order_date)
16    )
17    .order_by(
18        extract("year", Order.order_date),
19        extract("month", Order.order_date)
20    )
21    .subquery()
22 )
23
24 prev_rev = func.lag(monthly.c.revenue, 1).over(
25     order_by=[monthly.c.year, monthly.c.month]
26 ).label("prev_month_revenue")
27
28 stmt = select(monthly, prev_rev)
```

Listing 39: *Beispiel 10 – Python (Business Report)*

```
1 var monthly = context.Orders
2     .Where(o => o.OrderDate.Year == 2024
3         && o.Status != "pending")
4     .GroupBy(o => new {
5         Year = o.OrderDate.Year,
6         Month = o.OrderDate.Month
7     })
```

```

8      .Select(g => new {
9          g.Key.Year,
10         g.Key.Month,
11         Revenue      = g.Sum(o => o.Total),
12         OrderCount    = g.Count(),
13         UniqueCustomers = g.Select(o =>
14             o.CustomerId).Distinct().Count()
15     })
16     .OrderBy(x => x.Year)
17     .ThenBy(x => x.Month);
18
19 // LAG-Fenster: EF Core 8+
20 var withLag = monthly
21     .Select(m => new {
22         m.Year, m.Month, m.Revenue, m.OrderCount, m.UniqueCustomers,
23         PrevMonthRevenue = EF.Functions.Lag(m.Revenue, 1)
24             .Over()
25             .OrderBy(m.Year).ThenBy(m.Month)
26     });

```

Listing 40: *Beispiel 10 – C# (Business Report)*

UQTL-Normiertes SQL:

```

SELECT sub.year,
       sub.month,
       sub.revenue,
       sub.order_count,
       sub.unique_customers,
       LAG(sub.revenue, 1) OVER (
           ORDER BY sub.year ASC, sub.month ASC
       ) AS prev_month_revenue
FROM (
    SELECT EXTRACT(YEAR FROM t1.order_date) AS year,
           EXTRACT(MONTH FROM t1.order_date) AS month,
           SUM(t1.total) AS revenue,
           COUNT(t1.id) AS order_count,
           COUNT(DISTINCT t1.customer_id) AS unique_customers
    FROM orders AS t1
    WHERE EXTRACT(YEAR FROM t1.order_date) = 2024
          AND t1.status != 'pending'
    GROUP BY EXTRACT(YEAR FROM t1.order_date),
             EXTRACT(MONTH FROM t1.order_date)
    ORDER BY EXTRACT(YEAR FROM t1.order_date) ASC,
             EXTRACT(MONTH FROM t1.order_date) ASC
) AS sub

```

6 Konformitätsstufen

Der UQTL-Standard definiert vier Konformitätsstufen (Conformance Levels), die eine schrittweise Implementierung ermöglichen. Höhere Stufen schließen alle Anforderungen niedrigerer Stufen ein.

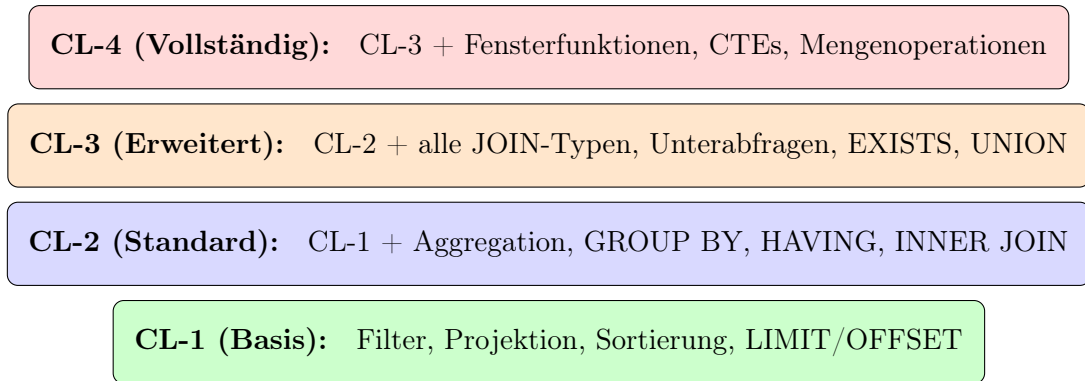


Abbildung 2: UQTL-Konformitätspyramide (CL-1 bis CL-4)

6.1 CL-1: Basiskonformität

Ein CL-1-konformes System muss die folgenden Operatoren vollständig und äquivalenzerhaltend unterstützen:

- **SCAN:** Zugriff auf einzelne Tabellen mit optionalem Alias
- **FILTER (WHERE):** Alle Vergleichsoperatoren ($=$, $\neq$, $<$, $\leq$, $>$, $\geq$), **IN**, **BETWEEN**, **LIKE**, **IS NULL**, **AND**, **OR**, **NOT**
- **PROJECT:** Spaltenselektion, Ausdrücke, **DISTINCT**
- **SORT:** **ORDER BY** **ASC/DESC**, mehrstufig
- **LIMIT:** **LIMIT** *n* **OFFSET** *m*

6.2 CL-2: Standardkonformität

Zusätzlich zu CL-1 müssen CL-2-konforme Systeme unterstützen:

- **GROUP _AGG:** **GROUP BY**, alle Standardaggregationen (**COUNT**, **SUM**, **AVG**, **MIN**, **MAX**)
- **FILTER (HAVING):** Aggregatprädikate nach **GROUP BY**
- **JOIN (INNER):** **INNER JOIN** ... **ON**, **NATURAL JOIN**
- **Arithmetische Ausdrücke** in Projektionen und Prädikaten

6.3 CL-3: Erweiterte Konformität

Zusätzlich zu CL-2:

- **JOIN (alle Typen):** LEFT/RIGHT/FULL OUTER JOIN, CROSS JOIN
- **SUBQUERY:** Korrelierte und nicht-korrelierte Unterabfragen, Scalar Subqueries
- **EXISTS / NOT EXISTS**
- **SET_OP:** UNION, UNION ALL
- **Skalare Funktionen:** Zeichenkettenfunktionen, Datumsfunktionen, mathematische Funktionen

6.4 CL-4: Vollständige Konformität

Zusätzlich zu CL-3:

- **WINDOW:** Alle ANSI-Fensterfunktionen (ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE, gleitende Aggregate)
- **WITH (CTEs):** Einfache und rekursive CTEs
- **SET_OP vollständig:** INTERSECT, EXCEPT
- **CASE-Ausdrücke**
- **Dialekt-Erweiterungen** (als optionale Zertifizierung)

Tabelle 11: *Konformitätsstufen-Übersicht*

Feature	CL-1	CL-2	CL-3	CL-4
Filter (WHERE)	✓	✓	✓	✓
Projektion	✓	✓	✓	✓
Sortierung	✓	✓	✓	✓
LIMIT/OFFSET	✓	✓	✓	✓
GROUP BY / Aggregation		✓	✓	✓
HAVING		✓	✓	✓
INNER JOIN		✓	✓	✓
OUTER JOINs			✓	✓
Unterabfragen			✓	✓
EXISTS			✓	✓
UNION / UNION ALL			✓	✓
Fensterfunktionen				✓
CTEs (WITH)				✓
INTERSECT / EXCEPT				✓
CASE-Ausdrücke				✓

7 Referenzimplementierung

7.1 Architektur einer UQTL-konformen Implementierung

Eine UQTL-konforme Implementierung besteht aus drei Schichten:

1. **Frontend-Parser:** Sprachspezifische Analyse und IQT-Aufbau (je ein Parser pro Quellsprache)
2. **IQT-Normalisierer:** Anwendung der Normalisierungsregeln (3.3)
3. **SQL-Codegenerator:** IQT-zu-SQL-Übersetzung nach Zieldialekt

7.2 Pseudocode des IQT-Builders (Python-Frontend)

```

1 class PythonIQTBuilder:
2     """Baut einen IQT-Baum aus einem SQLAlchemy Select-Statement."""

```

```

3
4 def build(self, stmt: Select) -> IQTNode:
5     node = self._visit_froms(stmt)           # SCAN / JOIN
6     node = self._visit_where(stmt, node)      # FILTER WHERE
7     node = self._visit_group_having(stmt, node) # GROUP_AGG +
8         HAVING
9     node = self._visit_select(stmt, node)      # PROJECT
10    node = self._visit_order(stmt, node)       # SORT
11    node = self._visit_limit(stmt, node)       # LIMIT
12    return node
13
14 def _visit_where(self, stmt, source):
15     if stmt.whereclause is not None:
16         pred = self._translate_predicate(stmt.whereclause)
17         return FilterNode(source=source, predicate=pred,
18                             position=FilterPosition.WHERE)
19     return source
20
21 def _translate_predicate(self, clause) -> IQTExpr:
22     if isinstance(clause, BinaryExpression):
23         left = self._translate_expr(clause.left)
24         right = self._translate_expr(clause.right)
25         op = self._map_operator(clause.operator)
26         return CompareExpr(left=left, op=op, right=right)
27     elif isinstance(clause, BooleanClauseList):
28         children = [self._translate_predicate(c) for c in
29                     clause.clauses]
30         logic = LogicOp.AND if clause.operator == operators.and_ \
31                 else LogicOp.OR
32         return LogicExpr(op=logic, operands=children)
33     # ... weitere Faelle

```

Listing 41: Python-Frontend: IQT-Builder (vereinfacht)

7.3 Pseudocode des SQL-Codegenerators

```

1 class AnsiSqlCodegen:
2     """Generiert ANSI SQL:2016 aus einem normalisierten IQT."""
3
4     def generate(self, node: IQTNode) -> str:
5         parts = self._collect(node)
6         return self._assemble(parts)
7
8     def _collect(self, node, parts=None) -> SqlParts:
9         if parts is None:
10             parts = SqlParts()
11

```

```

12         if isinstance(node, ScanNode):
13             parts.from_clause = f"{node.table} AS {node.alias}"
14
15         elif isinstance(node, FilterNode):
16             self._collect(node.source, parts)
17             pred_sql = self._gen_expr(node.predicate)
18             if node.position == FilterPosition.WHERE:
19                 parts.where_clauses.append(pred_sql)
20             else:
21                 parts.having_clauses.append(pred_sql)
22
23         elif isinstance(node, JoinNode):
24             self._collect(node.left, parts)
25             right_sql = f"{node.right.table} AS {node.right.alias}"
26             cond_sql = self._gen_expr(node.condition.predicate)
27             join_kw = JOIN_KEYWORDS[node.join_type]
28             parts.joins.append(f"{join_kw} {right_sql} ON {cond_sql}")
29
30         elif isinstance(node, GroupAggNode):
31             self._collect(node.source, parts)
32             parts.group_by = ", ".join(
33                 self._gen_expr(e) for e in node.group_by)
34             parts.select_items.extend(
35                 f"{self._gen_agg(a)} AS {a.alias}" for a in
36                 node.aggregates)
37
38         elif isinstance(node, ProjectNode):
39             self._collect(node.source, parts)
40             parts.distinct = node.distinct
41             parts.select_items = [
42                 self._gen_proj(item) for item in node.columns]
43
44         elif isinstance(node, SortNode):
45             self._collect(node.source, parts)
46             parts.order_by = ", ".join(
47                 self._gen_sort_item(s) for s in node.items)
48
49         elif isinstance(node, LimitNode):
50             self._collect(node.source, parts)
51             if node.count is not None:
52                 parts.limit = str(self._gen_expr(node.count))
53             if node.offset is not None:
54                 parts.offset = str(self._gen_expr(node.offset))
55
56         return parts
57
58     def _assemble(self, p: SqlParts) -> str:

```



```

58     distinct = "DISTINCT " if p.distinct else ""
59     sel = ", ".join(p.select_items) if p.select_items else "*"
60     sql = f"SELECT {distinct}{sel}\nFROM {p.from_clause}"
61     if p.joins:
62         sql += "\n" + "\n".join(p.joins)
63     if p.where_clauses:
64         sql += "\nWHERE " + "\n AND ".join(p.where_clauses)
65     if p.group_by:
66         sql += f"\nGROUP BY {p.group_by}"
67     if p.having_clauses:
68         sql += "\nHAVING " + "\n AND ".join(p.having_clauses)
69     if p.order_by:
70         sql += f"\nORDER BY {p.order_by}"
71     if p.limit:
72         sql += f"\nLIMIT {p.limit}"
73     if p.offset:
74         sql += f"\nOFFSET {p.offset}"
75     return sql

```

Listing 42: *SQL-Codegenerator: IQT nach ANSI SQL*

7.4 Konformitätstests

Eine UQTL-Referenzimplementierung muss eine Test-Suite bestehen, die für jede Konformitätsstufe die semantische Äquivalenz der Übersetzungen prüft:

```

1 class UQTLConformanceTest:
2
3     def assert_equivalent(self, py_stmt, java_iqt, cs_iqt,
4                           expected_sql):
5         """Prueft semantische Aequivalenz aller drei
6            Uebersetzungen."""
7         py_sql = PythonIQTBuilder().build(py_stmt)
8                 |> IQTNormalizer().normalize()
9                 |> AnsiSqlCodegen().generate()
10        java_sql = java_iqt
11                 |> IQTNormalizer().normalize()
12                 |> AnsiSqlCodegen().generate()
13        cs_sql = cs_iqt
14                 |> IQTNormalizer().normalize()
15                 |> AnsiSqlCodegen().generate()
16
17        assert py_sql == expected_sql, f"Python
18            divergiert:\n{py_sql}"
19        assert java_sql == expected_sql, f"Java
20            divergiert:\n{java_sql}"
21        assert cs_sql == expected_sql, f"C# divergiert:\n{cs_sql}"

```

Listing 43: *Konformitätstest-Schablone*

8 Diskussion

8.1 Stärken des UQTL-Standards

Der UQTL-Standard bietet gegenüber dem Status quo mehrere fundamentale Vorteile:

Verifikationsgrundlage Durch das definierte Normalformat können Query-Äquivalenzprüfungen vollständig automatisiert werden. Dies ermöglicht:

- Automatisierte Regressionstests bei Frameworkmigrationen
- Statische Analyse von Abfrage-Äquivalenz in CI/CD-Pipelines
- Formale Verifikation kritischer Datenzugriffspfade

Reduzierung kognitiver Last Entwickler, die in mehrsprachigen Teams arbeiten, können Abfragen in ihrer bevorzugten Sprache formulieren und sind dennoch garantiert, dass semantisch gleiche Abfragen zu identischen SQL-Anweisungen führen. Code-Reviews werden deutlich vereinfacht, da der Vergleich auf der Ebene des normalisierten SQL stattfinden kann.

Langzeitstabilität Frameworks evolvieren: SQLAlchemy hat mit Version 2.0 eine neue API eingeführt, Entity Framework hat mehrere inkompatible Generationen durchlaufen. Ein stabiler Standard abstrahiert von diesen Evolutionen.

8.2 Grenzen und Limitierungen

Semantisch nicht äquivalente Quellkonstrukte Nicht alle Quellsprachkonstrukte haben eindeutige SQL-Entsprechungen. Beispielsweise kann eine Python-Lambda-Funktion in einem `filter()`-Ausdruck beliebige Python-Logik enthalten, die kein SQL-Äquivalent hat. UQTL definiert für solche Fälle einen *Fallback-Mechanismus*: Der betreffende Ausdruck wird als `OPAQUE`-Knoten im IQT markiert und löst einen Normalisierungsfehler aus.

Dialektspezifische Funktionen Bestimmte Datenbankfunktionen (z. B. PostgreSQL-spezifisches `jsonb_agg()`, Oracle-spezifisches `CONNECT BY`) sind außerhalb von Erweiterungen nicht im Standard enthalten. Implementierungen, die diese benötigen, müssen einen Dialekt-Erweiterungsmechanismus implementieren.

Performanz-Implikationen Die Normalisierung erzeugt in einigen Fällen SQL-Ausdrücke, die semantisch korrekt, aber nicht optimal für alle Datenbankplaner sind.

Implementierungen sind ermutigt, nach der Normalisierung einen optionalen *Hint-Layer* anzuwenden.

Stream-API vs. Datenbankabfragen Java Streams operieren in-memory und unterstützen Operationen (z. B. `peek()`, `forEach()`, zustandsbehaftete Lambdas), die keine SQL-Entsprechung haben. UQTL richtet sich an den Subset der Stream-Operationen, der eine relationale Semantik trägt.

8.3 Erweiterungen und zukünftige Arbeit

Folgende Erweiterungen sind für zukünftige Versionen des Standards geplant:

1. **UQTL-E1 (NoSQL-Dialekte):** Erweiterung um MongoDB-Aggregationspipeline und Elasticsearch-DSL als zusätzliche Quellsprachen
2. **UQTL-E2 (TypeScript):** Unterstützung von TypeScript/Prisma als vierte Quellsprache
3. **UQTL-E3 (Streaming SQL):** Integration von Apache Flink SQL und Apache Spark SQL als Zieldialekte
4. **UQTL-E4 (Schema-Awareness):** Integration von Schemavalidierung und Typ-Prüfung in den Normalisierungsprozess
5. **UQTL-E5 (ML-gestützte Optimierung):** Automatische Query-Optimierung basierend auf historischen Ausführungsstatistiken

9 Zusammenfassung

Die vorliegende Arbeit hat den **Unified Query Translation Layer (UQTL)** vollständig spezifiziert: einen offenen Standard zur garantierten semantischen Äquivalenz von Query-Übersetzungen aus Python, Java (Stream API / JPA) und C#/.NET (LINQ) nach SQL.

Kernbeiträge Die wesentlichen Beiträge des Standards lassen sich wie folgt zusammenfassen:

1. **Formale Grundlage:** Die Definition des Intermediate Query Tree (IQT) als typisierter DAG mit vollständigen Knotenspezifikationen für alle 11 Knotentypen (SCAN, FILTER, PROJECT, JOIN, GROUP_AGG, SORT, LIMIT, WINDOW, SUBQUERY, SET_OP, WITH).

2. **Vollständige Übersetzungsregeln:** Für alle 9 Hauptoperatoren (Filter, Projektion, Sortierung, Limit, Aggregation, Joins, Fensterfunktionen, Unterabfragen, Mengenoperationen) wurden vollständige, formale Übersetzungsregeln von allen drei Quellsprachen in den IQT und vom IQT nach ANSI SQL:2016 definiert.
3. **10 Praxisbeispiele:** Anhand eines einheitlichen E-Commerce-Datenmodells wurden 10 vollständige Beispiele demonstriert, die von einfacher Filterung bis zu komplexen Geschäftsberichten mit CTEs und Fensterfunktionen reichen.
4. **Konformitätsstufen:** Vier Stufen (CL-1 bis CL-4) erlauben einen schrittweisen Aufbau konformer Implementierungen.
5. **Referenzimplementierungsrichtlinien:** Architektur und Pseudocode einer konformen Implementierung wurden bereitgestellt.

Bedeutung für die Praxis Der UQTL-Standard schließt eine fundamentale Lücke in der polyglotten Softwareentwicklung: Erstmals können Teams aus Python-, Java- und C#-Entwicklern formal garantieren, dass ihre Datenbankzugriffe semantisch äquivalent sind – unabhängig von Sprache, Framework-Version oder persönlichem Codingstil.

Leitprinzip des Standards:

Dieselbe Datenabfrage, formuliert in unterschiedlichen Sprachen, erzeugt unter UQTL stets dieselbe SQL-Anweisung. Die Sprache ist ein Implementierungsdetail – die Semantik ist der Standard.

A Vollständige IQT-Grammatik (EBNF)

Die folgende EBNF-Grammatik definiert die vollständige Syntax des Intermediate Query Tree.

```
1  IQT          ::= QueryNode
2
3  QueryNode    ::= ScanNode
4                | FilterNode
5                | ProjectNode
6                | JoinNode
7                | GroupAggNode
8                | SortNode
9                | LimitNode
10               | WindowNode
11               | SubqueryNode
12               | SetOpNode
13               | WithNode
14
15 ScanNode      ::= 'SCAN' '{' 'table' ':' QualName
16                  ('alias' ':' Ident)?
17                  ('hint' ':' IndexHint)?
18                  '}',
19
20 FilterNode    ::= 'FILTER' '{' 'source'      ':' QueryNode
21                  'predicate' ':' Expr
22                  'position'  ':' ('WHERE' | 'HAVING')
23                  '}',
24
25 ProjectNode   ::= 'PROJECT' '{' 'source'      ':' QueryNode
26                  'columns'   ':' ProjItem+
27                  'distinct'  ':' Bool
28                  '}',
29
30 ProjItem      ::= 'Star' | 'Expr' '{' 'expr'  ':' Expr ('alias' ':'
31                  Ident)? '}',
32
33 JoinNode      ::= 'JOIN' '{' 'left'          ':' QueryNode
34                  'right'           ':' QueryNode
35                  'type'            ':' JoinType
36                  'condition'       ':' JoinCond
37                  '}',
38
39 JoinType      ::= 'INNER' | 'LEFT_OUTER' | 'RIGHT_OUTER'
40                  | 'FULL_OUTER' | 'CROSS'
41
42 JoinCond      ::= 'ON'      '{' 'predicate' ':' Expr '}'
```

```

42         | 'USING' '{ 'columns' ':' Ident+ '}'
43         | 'NATURAL'
44
45 GroupAggNode ::= 'GROUP_AGG' '{ 'source' ':' QueryNode
46                'groupBy' ':' Expr*
47                'aggregates' ':' AggItem+
48                ('having' ':' Expr)?
49                '}',
50
51 AggItem      ::= '{ 'func' ':' AggFunc
52                'arg' ':' (Expr | 'STAR')
53                'distinct' ':' Bool
54                ('alias' ':' Ident)?
55                '}',
56
57 AggFunc      ::= 'COUNT' | 'SUM' | 'AVG' | 'MIN' | 'MAX'
58                | 'STDDEV' | 'VAR' | 'STRING_AGG' | 'ARRAY_AGG'
59
60 SortNode     ::= 'SORT' '{ 'source' ':' QueryNode
61                'items' ':' SortItem+
62                '}',
63
64 SortItem     ::= '{ 'expr' ':' Expr
65                'direction' ':' ('ASC' | 'DESC')
66                'nulls' ':' ('NULLS_FIRST' | 'NULLS_LAST'
67                | 'DEFAULT')
68                '}',
69
70 LimitNode    ::= 'LIMIT' '{ 'source' ':' QueryNode
71                'count' ':' (Expr | 'NULL')
72                'offset' ':' (Expr | 'NULL')
73                '}',
74
75 WindowNode   ::= 'WINDOW' '{ 'source' ':' QueryNode
76                'definitions' ':' WindowDef+
77                '}',
78
79 WindowDef    ::= '{ 'alias' ':' Ident
80                ('partitionBy' ':' Expr*)?
81                ('orderBy' ':' SortItem*)?
82                ('frame' ':' WindowFrame)?
83                'func' ':' WindowFunc
84                'resultAlias' ':' Ident
85                '}',
86
87 WindowFrame  ::= '{ 'unit' ':' ('ROWS' | 'RANGE')
88                'start' ':' FrameBound

```

```

88         'end'      ':' FrameBound
89     '}',
90
91 FrameBound      ::= 'UNBOUNDED_PRECEDING' | 'CURRENT_ROW'
92                 | 'UNBOUNDED_FOLLOWING'
93                 | 'PRECEDING' '{ Expr }'
94                 | 'FOLLOWING' '{ Expr }'
95
96 SetOpNode       ::= 'SET_OP' '{ 'left' ':' QueryNode
97                   'right' ':' QueryNode
98                   'op'      ':' ('UNION' | 'UNION_ALL'
99                                   | 'INTERSECT' | 'EXCEPT')
100                 '}',
101
102 WithNode        ::= 'WITH' '{ 'ctes'  ':' CTEDef+
103                   'query' ':' QueryNode
104                 '}',
105
106 CTEDef          ::= '{ 'name'      ':' Ident
107                   'recursive' ':' Bool
108                   'query'      ':' QueryNode
109                 '}',
110
111 Expr            ::= CompareExpr | LogicExpr | InExpr | BetweenExpr
112                 | LikeExpr | NullExpr | ExistsExpr | CaseExpr
113                 | Literal | ColumnRef | FuncCall | ArithExpr |
114                 SubqueryExpr
115
116 CompareExpr     ::= '{ 'left' ':' Expr 'op' ':' CmpOp 'right' ':'
117                   Expr '}'
118 CmpOp           ::= 'EQ' | 'NEQ' | 'LT' | 'GT' | 'LTE' | 'GTE'
119 LogicExpr       ::= '{ 'op' ':' ('AND' | 'OR' | 'NOT') 'operands' ':'
120                   Expr+ '}'
121 InExpr          ::= '{ 'value' ':' Expr 'list' ':' (Expr+ |
122                   SubqueryNode)
123                   'negated' ':' Bool '}'
124 BetweenExpr     ::= '{ 'value' ':' Expr 'lo' ':' Expr 'hi' ':' Expr
125                   'negated' ':' Bool '}'
126 LikeExpr        ::= '{ 'value' ':' Expr 'pattern' ':' StringLit
127                   ('escape' ':' Char)? 'negated' ':' Bool '}'
128 NullExpr        ::= '{ 'value' ':' Expr 'negated' ':' Bool '}'
129 ExistsExpr      ::= '{ 'subquery' ':' SubqueryNode 'negated' ':' Bool
130                   '}'

```

Listing 44: *IQT-Grammatik in EBNF*

B Operatormapping-Referenztable

Tabelle 12: Vollständige Operatormapping-Referenz (IQT ↔ Quellsprachen)

IQT-Konstrukt	Python (SA 2.0)	Java (JPA/QueryDSL)	C# (LINQ)
SCAN (table)	<code>select(Model)</code>	<code>cq.from(Model.class)</code>	<code>context.Models</code>
SCAN (alias)	<code>aliased(Model, name="ä")</code>	<code>Root<M> a = cq.from(M.class)</code>	<code>- (implizit)</code>
FILTER (=)	<code>.where(M.col == v)</code>	<code>.where(cb.equal(r.get("col"),v))</code>	<code>.Where(x => x.Col == v)</code>
FILTER (!=)	<code>.where(M.col != v)</code>	<code>.where(cb.notEqual(...))</code>	<code>.Where(x => x.Col != v)</code>
FILTER (LIKE)	<code>.where(M.col.like("%x%"))</code>	<code>.where(cb.like(r.get("col"),"%x%"))</code>	<code>.Where(x => EF.Functions.Like(x.Col,"%x%"))</code>
FILTER (IS NULL)	<code>.where(M.col == None)</code>	<code>.where(cb.isNull(r.get("col")))</code>	<code>.Where(x => x.Col == null)</code>
FILTER (IN list)	<code>.where(M.col.in_(list))</code>	<code>.where(r.get("col").in(list))</code>	<code>.Where(x => list.Contains(x.Col))</code>
FILTER (IN subq)	<code>.where(M.col.in_(subq))</code>	<code>.where(r.get("col").in(subq))</code>	<code>.Where(x => subq.Contains(x.Col))</code>
FILTER (BETWEEN)	<code>.where(M.col.between(a,b))</code>	<code>.where(cb.between(r.get("col"),a,b))</code>	<code>.Where(x => x.Col >= a && x.Col <= b)</code>
FILTER (EXISTS)	<code>.where(exists(subq))</code>	<code>.where(cb.exists(subq))</code>	<code>.Where(x => subq.Any(...))</code>
PROJECT (cols)	<code>select(M.a, M.b)</code>	<code>cq.multiselect(r.get("ä"),r.get("b"))</code>	<code>.Select(x => new{x.A,x.B})</code>
PROJECT (expr)	<code>select(func.upper(M.name))</code>	<code>cq.select(cb.upper(r.get("name")))</code>	<code>.Select(x => x.Name.ToUpper())</code>
PROJECT (DISTINCT)	<code>select(M.col).distinct()</code>	<code>cq.select(r.get("col")).distinct(true)</code>	<code>.Select(x => x.Col.Distinct())</code>
SORT (ASC)	<code>.order_by(M.col)</code>	<code>.orderBy(cb.asc(r.get("col")))</code>	<code>.OrderBy(x => x.Col)</code>
SORT (DESC)	<code>.order_by(desc(M.col))</code>	<code>.orderBy(cb.desc(r.get("col")))</code>	<code>.OrderByDescending(x => x.Col)</code>
SORT (multi)	<code>.order_by(M.a, desc(M.b))</code>	<code>.orderBy(cb.asc(...), cb.desc(...))</code>	<code>.OrderBy(x=>x.A).ThenByDescending(x=>x.B)</code>
LIMIT	<code>.limit(n)</code>	<code>.setMaxResults(n)</code>	<code>.Take(n)</code>
OFFSET	<code>.offset(m)</code>	<code>.setFirstResult(m)</code>	<code>.Skip(m)</code>
JOIN (INNER)	<code>.join(B, A.id==B.a_id)</code>	<code>r.join("bs", JoinType.INNER)</code>	<code>join b in Bs on a.Id equals b.AId</code>
JOIN (LEFT)	<code>.outerjoin(B, A.id==B.a_id)</code>	<code>r.join("bs", JoinType.LEFT)</code>	<code>join b ... into g from b in g.DefaultIfEmpty()</code>
JOIN (CROSS)	<code>select_from(A, B)</code>	<code>cq.from(A.class); cq.from(B.class)</code>	<code>from a in As from b in Bs ...</code>
GROUP BY	<code>.group_by(M.col)</code>	<code>.groupBy(r.get("col"))</code>	<code>.GroupBy(x => x.Col)</code>

Fortsetzung auf nächster Seite

Fortsetzung von Tabelle 12

IQT-Konstrukt	Python	Java	C#
COUNT(*)	<code>func.count(*)</code>	<code>cb.count(root)</code>	<code>.Count()</code>
SUM	<code>func.sum(M.col)</code>	<code>cb.sum(r.get("col"))</code>	<code>.Sum(x => x.Col)</code>
AVG	<code>func.avg(M.col)</code>	<code>cb.avg(r.get("col"))</code>	<code>.Average(x => x.Col)</code>
MIN / MAX	<code>func.min/max(M.col)</code>	<code>cb.min/max(r.get("col"))</code>	<code>.Min/.Max(x => x.Col)</code>
HAVING	<code>.having(func.count(*) > n)</code>	<code>.having(cb.gt(cb.count(r), n))</code>	<code>.Where(g => g.Count() > n)</code>
UNION	<code>q1.union(q2)</code>	<code>(JPQL UNION)</code>	<code>q1.Union(q2)</code>
UNION ALL	<code>q1.union_all(q2)</code>	<code>(JPQL UNION ALL)</code>	<code>q1.Concat(q2)</code>
ROW_NUMBER()	<code>func.row_number().over(...)</code>	<code>cb.rowNumber().over(...)</code>	<code>EF.Functions.RowNumber(...)</code>
RANK()	<code>func.rank().over(...)</code>	<code>cb.rank().over(...)</code>	<code>EF.Functions.Rank(...)</code>
LAG(col, n)	<code>func.lag(M.col, n).over(...)</code>	<code>cb.lag(r.get("col"), n).over(...)</code>	<code>EF.Functions.Lag(x.Col, n)...</code>
CTE	<code>subq.cte("name")</code>	<code>(Blaze-Persistence CTE)</code>	<code>(EF Core 7+ native CTEs)</code>

Literatur

- [1] E. F. Codd, *A Relational Model of Data for Large Shared Data Banks*, Communications of the ACM, 13(6):377–387, 1970.
- [2] ISO/IEC 9075-1:2016, *Information technology – Database languages – SQL – Part 1: Framework (SQL/Framework)*, International Organization for Standardization, 2016.
- [3] M. Bayer et al., *SQLAlchemy – The Python SQL Toolkit and Object Relational Mapper*, <https://docs.sqlalchemy.org>, Version 2.0, 2023.
- [4] G. King, C. Bauer, et al., *Hibernate ORM Documentation*, <https://docs.jboss.org/hibernate/orm/>, Version 6.4, 2023.
- [5] Microsoft Corporation, *Entity Framework Core Documentation*, <https://learn.microsoft.com/en-us/ef/core/>, 2023.
- [6] Microsoft Corporation, *Language Integrated Query (LINQ) – C# Programming Guide*, <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>, 2023.
- [7] Oracle Corporation, *java.util.stream – Stream API*, Java SE 17 API Documentation, <https://docs.oracle.com/en/java/documentation/>, 2021.
- [8] T. Kallioikoski, *Querydsl – Unified Queries for Java*, <http://querydsl.com>, Version 5.0, 2023.
- [9] L. Waehner, *jOOQ – The Best Way to Write SQL in Java*, <https://www.jooq.org/doc/>, Version 3.18, 2023.
- [10] C. J. Date, H. Darwen, *A Guide to the SQL Standard*, 4th Edition, Addison-Wesley, 1997.
- [11] H. Garcia-Molina, J. D. Ullman, J. Widom, *Database Systems: The Complete Book*, 2nd Edition, Pearson Prentice Hall, 2009.
- [12] A. V. Aho, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, Addison-Wesley, 1986.
- [13] P. Wadler, *Comprehending Monads*, Proceedings of the 1990 ACM Conference on Lisp and Functional Programming, pp. 61–78, 1990.
- [14] P. Buneman, L. Libkin, D. Suciu, V. Tannen, L. Wong, *Comprehension Syntax*, SIGMOD Record, 23(1):87–96, 1994.